

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 3 : 딥러닝

DAY2. 딥러닝 네트워크 신경망 알고리즘 계층

구조

목 차

1. 인공신경망과 퍼셉트론		3
2. 논리회로와 다층 퍼셉트론		7
3. 딥러닝 신경망 계층구조		21
4. 활성화 함수		30

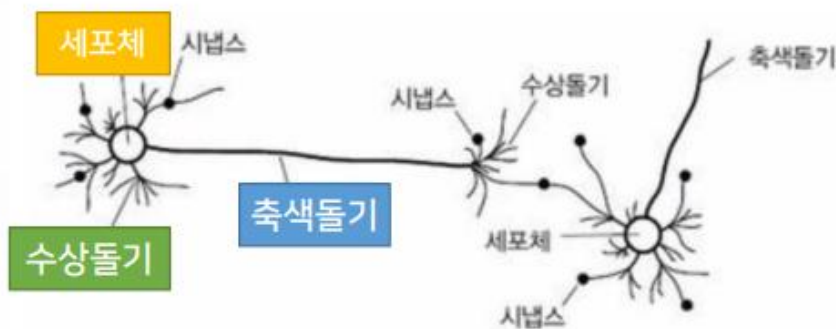
1. 인공신경망과 퍼셉트론

1) 뉴런(신경세포)이란?

(1) 뉴런(신경세포)

■ 뉴런(신경세포)

- 신경계를 구성하는 기본 세포 단위
- 신경계의 모든 작용은 뉴런의 상호작용으로 인해 발생
- 시냅스 구조를 통해 신호 전달



- 인간의 뇌에는 뉴런이 약 1000억개, 시냅스는 약 100조개가 있음

뉴런은 다음과 같은 역할을 수행한다.

수상돌기(Dendrite)

다른 뉴런으로부터 신호를 입력받는다.

세포체(Cell Body)

입력된 신호를 계산한다.

축색돌기(Axon)

계산 결과를 다음 뉴런으로 전달한다.

PyTorch 관점에서 보면

```
x = torch.tensor([1.0, 2.0])
```

입력 데이터는 수상돌기에 해당한다.

```
y = model(x)
```

계산 과정은 세포체 역할이다.

```
output
```

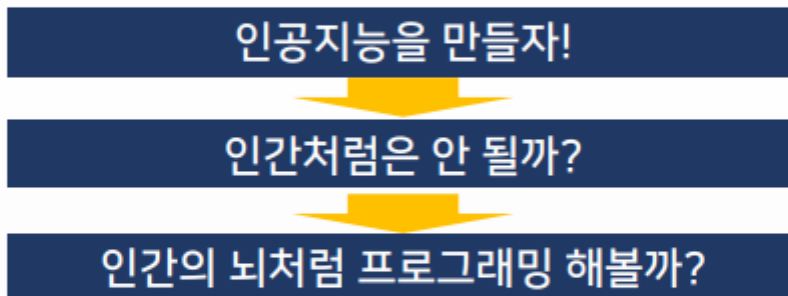
은 축색돌기를 통해 전달되는 출력 신호에 해당한다.

2) 인공지능망이란?

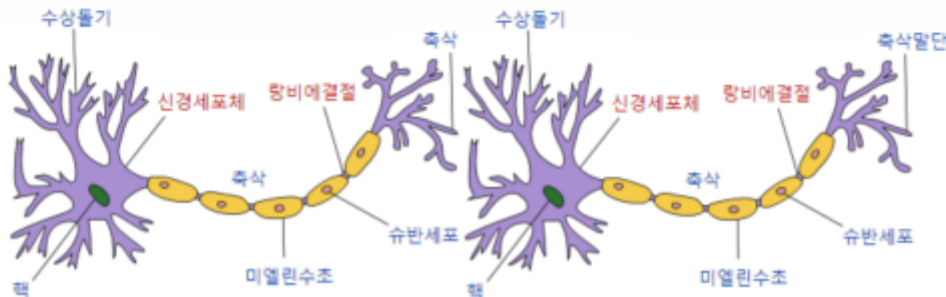
(1) 인공지능망

- 기계학습의 알고리즘 중 하나로 인간의 뇌에서 영감을 얻어 만들어진 알고리즘
- 인간 뇌의 신경세포(뉴런)를 추상화하여 비슷하게 만든 인공지능망

(2) 인공지능망 등장 배경



- 뉴런의 구조를 본떠서 추상화해서 만들게 됨



연구자들은 인간의 뇌를 관찰하면서 다음과 같은 질문을 하게 되었다.

"컴퓨터도 인간처럼 학습할 수 있을까?"

이 질문에서 인공지능망이 탄생하였다.

인공지능망은 인간 뉴런을 수학적으로 단순화한 모델이다.

PyTorch에서는 이러한 뉴런을 다음과 같이 표현한다.

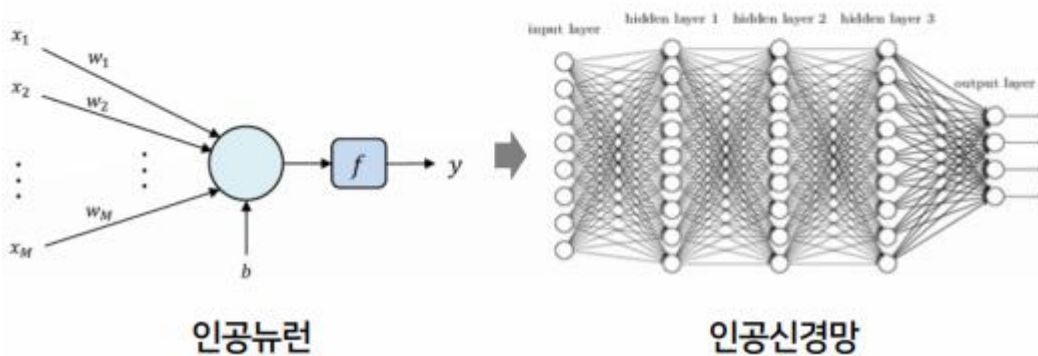
```
import torch
import torch.nn as nn

neuron = nn.Linear(2,1)
```

3) 퍼셉트론(인공뉴런)

(1) 퍼셉트론(인공뉴런)

- 인공신경망 모형 중 하나로 뉴런을 참고하여 만듦
- 현재 딥러닝의 기원
- 여러 개의 입력을 받을 수 있고, 하나의 출력을 내보낼 수 있음
- 다음의 뉴런으로 정보를 전달할지 여부를 결정함



퍼셉트론은 인공뉴런을 수학적으로 구현한 모델이다.

입력값

$$x_1, x_2, \dots, x_n$$

과 가중치

$$w_1, w_2, \dots, w_n$$

를 곱하여 합산한다

퍼셉트론 수식

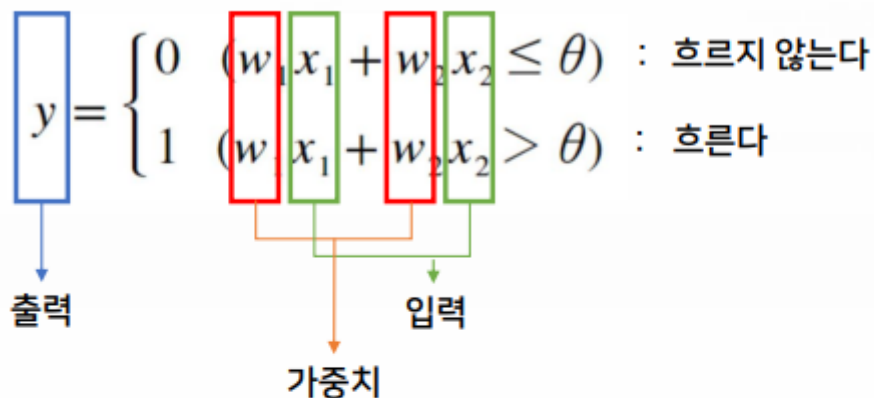
$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

PyTorch 구현

```
import torch.nn as nn
model = nn.Linear(2,1)
```

3) 퍼셉트론(인공뉴런)

(1) 퍼셉트론(인공뉴런)



가중치가 클수록?

- 입력이 결과에 많은 영향을 미침
- 입력이 전체 인공신경망에서 중요한 역할을 함

▪ θ 를 넘겨 식을 정리하고 일반화하면...

입력 * 가중치 + 편향 > 0 : 흐른다

입력 * 가중치 + 편향 ≤ 0 : 흐르지 않는다

가중치(Weight)는 입력의 중요도를 의미한다.

예를 들어, weight = [[0.8, 0.1]] 이라면

첫 번째 입력이 두 번째 입력보다 8 배 중요하다고 볼 수 있다.

편향(Bias)은 기본값이다.

bias = [0.5] 는 모든 입력이 0 이어도 출력이 0.5 만큼 이동한다는 의미이다.

PyTorch 확인

```
print(model.weight)
print(model.bias)
```

2. 논리회로와 다층 퍼셉트론

1) 퍼셉트론으로 구현하는 논리회로

(1) 논리회로 : 입력 2개(x_1, x_2), 출력 1개(1 또는 0)

▪ 퍼셉트론으로 구현 가능

x_1	x_2	y	x_1	x_2	y	x_1	x_2	y
0	0	0	0	0	1	0	0	0
1	0	0	1	0	1	1	0	1
0	1	0	0	1	1	0	1	1
1	1	1	1	1	0	1	1	1

AND NAND OR

- $x_1 * w_1 + x_2 * w_2 + b > 0$ 을 만족하는 w_1, w_2, b 찾기

AND : [0.5, 0.5, -0.7], [0.6, 0.7, -0.8], [0.3, 0.3, -0.5], [1.0, 1.0, -1.5] ...

NAND : [-0.2, -0.2, 0.3] ...

OR : [0.7, 0.7, -0.4] ...

퍼셉트론은 간단한 논리회로를 구현할 수 있다.

대표적으로

- AND
- OR
- NAND

게이트가 있다.

1. AND 게이트 (PyTorch)

AND 게이트 구현 : 입력이 모두 1 일 때만 1 출력

```
import torch
```

```
def AND(x1, x2):
```

```
    # 입력값을 Tensor 로 생성
```

```
    x = torch.tensor([x1, x2], dtype=torch.float32)
```

```
    # AND 를 만족하는 가중치
```

```
    w = torch.tensor([0.5, 0.5])
```

```
    # 편향(Bias)
```

```
    b = -0.7
```

```
    # 입력 * 가중치 + 편향 계산
```

```
    tmp = torch.sum(x * w) + b
```

```
    # 임계값 판단
```

```
    if tmp <= 0:
```

```
        return 0
```

```
    else:
```

```
        return 1
```

```
print(AND(0,0))
```

```
print(AND(0,1))
```

```
print(AND(1,0))
```

```
print(AND(1,1))
```

출력

0

0

0

1

2. NAND 게이트 (PyTorch)

```
# NAND 게이트 구현 : AND 결과를 반전
import torch

def NAND(x1, x2):
    x = torch.tensor([x1, x2], dtype=torch.float32)

    # 음수 가중치 사용
    w = torch.tensor([-0.2, -0.2])

    b = 0.3

    tmp = torch.sum(x * w) + b

    if tmp <= 0:
        return 0
    else:
        return 1

print(NAND(0,0))
print(NAND(0,1))
print(NAND(1,0))
print(NAND(1,1))
```

출력

```
1
1
1
0
```

3. OR 게이트 (PyTorch)

```
# OR 게이트 구현 : 하나라도 1 이면 출력 1
import torch

def OR(x1, x2):
    x = torch.tensor([x1, x2], dtype=torch.float32)

    w = torch.tensor([0.7, 0.7])

    b = -0.4

    tmp = torch.sum(x * w) + b

    if tmp <= 0:
        return 0
    else:
        return 1

print(OR(0,0))
print(OR(0,1))
print(OR(1,0))
print(OR(1,1))
```

출력

```
0
1
1
1
```

4. 무작위 Weight 탐색 함수

```
# 학습 없이 랜덤 Weight 탐색 : 논리회로를 만족하는 Weight 찾기
import torch

def check(x1, x2, y):
    while True:
        cnt = 0

        # 정규분포에서 랜덤 Weight 생성
        w1 = torch.randn(1).item()
        w2 = torch.randn(1).item()
        b = torch.randn(1).item()

        for i in range(len(x1)):
            x = torch.tensor([x1[i], x2[i]], dtype=torch.float32)
            w = torch.tensor([w1, w2])
            tmp = torch.sum(x * w) + b

            if tmp <= 0:
                result = 0
            else:
                result = 1

            if y[i] == result:
                cnt += 1

        # 4 개 데이터 모두 정답이면 종료
        if cnt == 4:
            print("w1 =", w1)
            print("w2 =", w2)
            print("b  =", b)

            break
```

5. AND 게이트 Weight 자동 탐색

```
# AND 게이트용 Weight 탐색
def AND_SEARCH():
    x1 = [0,1,0,1]
    x2 = [0,0,1,1]
    y = [0,0,0,1]

    check(x1, x2, y)

print("AND")
AND_SEARCH()
```

6. NAND, OR 게이트 Weight 자동 탐색

```
# NAND 게이트 Weight 탐색
def NAND_SEARCH():
    x1 = [0,1,0,1]
    x2 = [0,0,1,1]
    y = [1,1,1,0]

    check(x1, x2, y)

# OR 게이트 Weight 탐색
def OR_SEARCH():
    x1 = [0,1,0,1]
    x2 = [0,0,1,1]
    y = [0,1,1,1]

    check(x1, x2, y)

print("\nNAND")
NAND_SEARCH()
```

```
print("\nOR")
OR_SEARCH()
```

위의 예제는 **학습(Training)**이 없는 단순 랜덤 탐색입니다.

실제 딥러닝 방식으로 구현하려면 다음과 같이 합니다.

```
import torch
import torch.nn as nn
import torch.optim as optim

# 단일 퍼셉트론
model = nn.Linear(2, 1)
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(
    model.parameters(),
    lr=0.01
)

X = torch.tensor([
    [0.,0.],
    [0.,1.],
    [1.,0.],
    [1.,1.]
])

y = torch.tensor([
    [0.],
    [0.],
    [0.],
    [1.]
])

for epoch in range(1000):
    pred = model(X)
```

```
loss = criterion(pred, y)
optimizer.zero_grad()
loss.backward()
optimizer.step()

if epoch % 100 == 0:
    print(epoch, loss.item())
```

이 코드는 **AND 게이트의 Weight(w)와 Bias(b)**를 역전파를 통해 자동 학습하는 현대적인 PyTorch 구현 방식입니다.

2) 단층 퍼셉트론의 한계

(1) XOR 게이트 구현하기

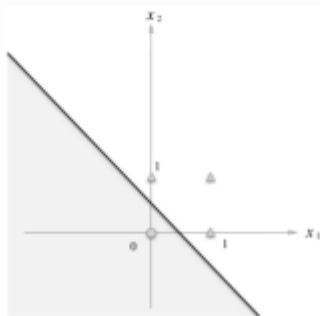
- 만족하는 w_1, w_2, b 찾기

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

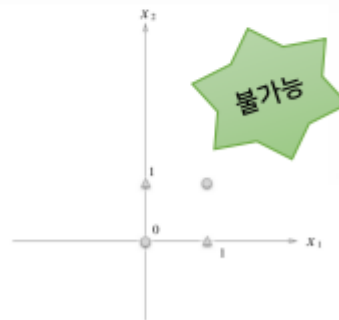
```
In [*]: def XOR():  
    x1 = [0, 1, 0, 1]  
    x2 = [0, 0, 1, 1]  
    y = [0, 1, 1, 0]  
    check(x1, x2, y)  
  
print('XOR')  
XOR()
```

$$\begin{aligned} x_1 * w_1 + x_2 * w_2 + b &> 0 \rightarrow 1 \\ x_1 * w_1 + x_2 * w_2 + b &\leq 0 \rightarrow 0 \end{aligned}$$

(2) 동그라미와 세모를 분리할 수 있는 하나의 직선 그리기



OR



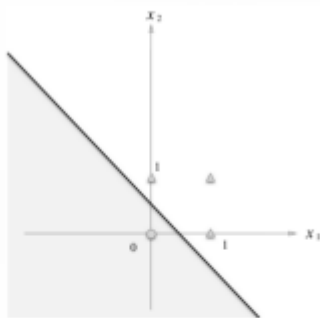
XOR

- 단층 퍼셉트론은 1차함수이고 직선으로만 표현 가능함

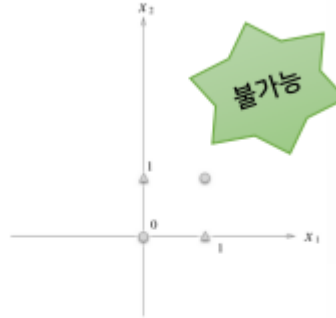
XOR 는 하나의 직선으로 구분할 수 없다.
즉, 단층 퍼셉트론으로 해결이 불가능하다.

2) 단층 퍼셉트론의 한계

(2) 동그라미와 세모를 분리할 수 있는 하나의 직선 그리기



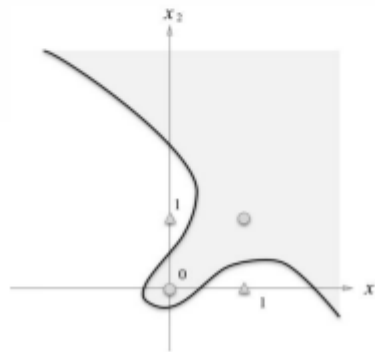
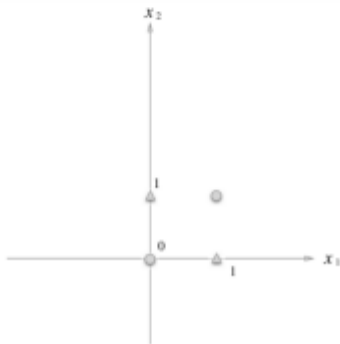
OR



XOR

- 단층 퍼셉트론은 1차함수이고 직선으로만 표현 가능함

(3) 한 개의 퍼셉트론으로 표현 불가능



단층 퍼셉트론은

$$y = w_1x_1 + w_2x_2 + b$$

형태이다. 즉, 결정경계가 직선이다.

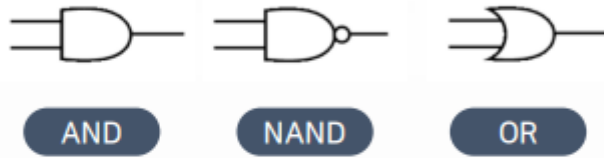
복잡한 데이터는 곡선 경계가 필요하다.

따라서 XOR 문제를 해결할 수 없다.

3) 다층 퍼셉트론

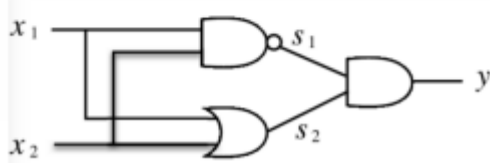
(1) XOR 게이트 만들기

- 기존 AND, OR, NAND 를 조합하여 생성



x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

- 0층 : 입력 x_1, x_2
- 1층 : NAND, OR 계산
- 2층 : 1층의 결과를 입력으로 AND



x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0

XOR 문제를 해결하기 위해 은닉층(Hidden Layer)을 추가한다.

구조

입력층

↓

은닉층

↓

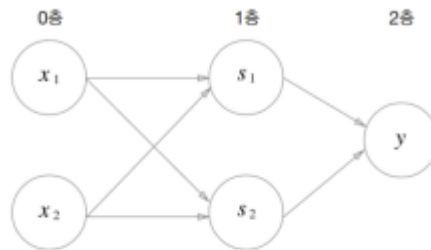
출력층

3) 다층 퍼셉트론

(2) XOR 게이트 만들기(파이썬 코드)

```
def XOR(x1, x2):  
    s1 = NAND(x1, x2)  
    s2 = OR(x1, x2)  
    y = AND(s1, s2)  
    return y  
input_data = [(0,0),(1,0),(0,1),(1,1)]  
  
print('XOR')  
for i in input_data:  
    print(XOR(i[0],i[1]))
```

```
XOR  
0  
1  
1  
0
```



(3) 논리회로 퍼셉트론으로 구현



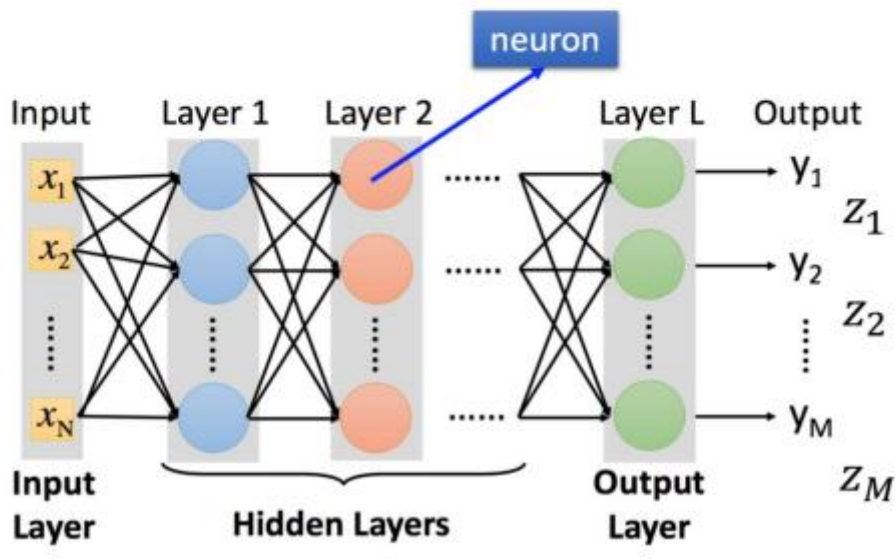
4) 인공지능망과 딥러닝

(1) 인공지능망과 딥러닝

- 실제 일상생활에선 논리회로(AND, OR 등), 퍼셉트론으로 해결할 수 있는 문제는 거의 없음
- 더 복잡한 문제를 해결하기 위해 층을 쌓다보니 인간의 뇌처럼 복잡한 인공지능망이 탄생함

(2) 딥러닝

- Deep은 '깊다' 라는 뜻으로 인공지능망의 층이 '아주 깊게 쌓여 있다' 라는 것에서 유래됨



현실 문제는 XOR 보다 훨씬 복잡하다.

예

- 얼굴 인식
- 음성 인식
- 자연어 처리
- 자율주행

이를 해결하기 위해 은닉층을 여러 개 쌓은 구조가 등장하였다.

이를 Deep Neural Network 라고 부른다.

3. 딥러닝 신경망 계층구조

딥러닝 신경망 알고리즘은 입력 데이터에서 패턴을 찾기 위해 여러 개의 인공뉴런을 층(layer)으로 연결한 학습 구조입니다.

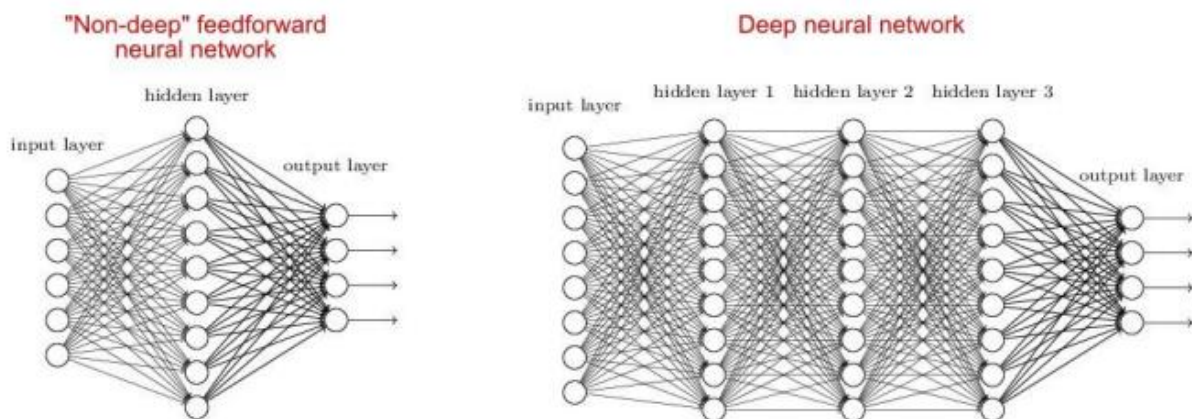
가장 기본 흐름은 다음과 같습니다.

입력 데이터가 들어오면 각 뉴런은 입력값에 가중치 w 를 곱하고 편향 b 를 더한 뒤, 활성화 함수를 통과시켜 다음 층으로 전달합니다.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$$a = f(z)$$

여기서 z 는 뉴런 내부 계산값이고, a 는 활성화 함수를 지난 출력값입니다.



$$y = w_2(\text{act}(w_1 * \text{input} + b_1)) + b_2$$

$$y = w_4(\text{act}(w_3(\text{act}(w_2(\text{act}(w_1 * \text{input} + b_1)) + b_2)) + b_3)) + b_4$$

$$Y = W \cdot X + b$$

$$\begin{bmatrix} x_{00} & x_{01} & \dots & x_{0n} \\ x_{10} & x_{11} & \dots & x_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m0} & x_{m1} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} w_{00} & w_{01} & w_{02} & \dots & w_{0l} \\ w_{10} & w_{11} & \dots & \dots & w_{1l} \\ w_{20} & \dots & \dots & \dots & \dots \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ w_{n0} & w_{n1} & \dots & \dots & w_{nl} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_m \end{bmatrix} = Y$$

$m \times n$ $n \times l$ $m \times 1$

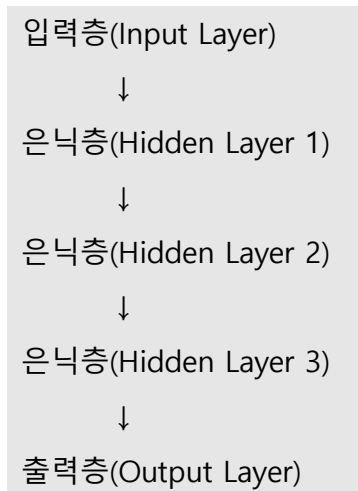
$$y = \text{act}(wx + b)$$

$$= \text{activation} \left(\left[\begin{array}{c} wx + b \end{array} \right] \right)$$

$m \times 1$

1. 딥러닝 신경망의 기본 계층구조

딥러닝 신경망은 보통 다음 구조를 가집니다.



입력층

입력층은 데이터를 모델에 넣는 부분입니다.

예를 들어 집값 예측 모델이라면 입력값은 다음과 같습니다.

방 개수, 면적, 위치, 건축연도, 역세권 여부

이미지 분류 모델이라면 입력값은 픽셀값입니다.

은닉층

은닉층은 실제로 특징을 학습하는 부분입니다.

첫 번째 은닉층은 단순한 특징을 학습합니다.

예를 들어 이미지에서는 선, 점, 밝기, 색상 변화 를 학습합니다.

두 번째 은닉층은 조금 더 복잡한 특징을 학습합니다.

눈, 코, 입, 모서리, 질감

세 번째 이상 깊은 층은 더 추상적인 특징을 학습합니다.

얼굴, 자동차, 고양이, 사람 형태

즉, 딥러닝은 층이 깊어질수록 단순한 특징 → 복잡한 특징 → 의미 있는 개념으로 발전합니다.

출력층

출력층은 최종 결과를 내보내는 부분입니다.

문제 유형에 따라 출력층 구조가 달라집니다.

문제 유형	출력층 예
집값 예측	숫자 1 개
이진 분류	0 또는 1
다중 분류	클래스 개수만큼 출력
이미지 생성	이미지 픽셀값
문장 생성	다음 단어 확률

2. 딥러닝 학습 알고리즘의 전체 흐름

딥러닝 학습은 다음 순서로 진행됩니다.

1. 입력 데이터 준비
2. 순전파(Forward Propagation)
3. 예측값 계산
4. 손실값(Loss) 계산
5. 역전파(Backpropagation)
6. 가중치와 편향 업데이트
7. 반복 학습

3. 순전파(Forward Propagation)

순전파는 입력 데이터가 앞에서 뒤로 이동하면서 예측값을 만드는 과정입니다.

예를 들어 입력값이 다음과 같다고 하겠습니다.

x1 = 방 개수

x2 = 면적

x3 = 건축연도

각 입력은 가중치와 곱해지고 편향이 더해집니다.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

그 다음 활성화 함수를 통과합니다.

$$a = \text{ReLU}(z)$$

이렇게 계산된 값이 다음 층으로 전달됩니다.

4. 활성화 함수의 역할

활성화 함수는 신경망에 **비선형성**을 추가합니다.

활성화 함수가 없으면 아무리 층을 많이 쌓아도 결국 하나의 선형식과 같습니다.

대표적인 활성화 함수는 다음과 같습니다.

함수	특징	주 사용처
Sigmoid	0~1 출력	이진 분류 출력층
Tanh	-1~1 출력	RNN 일부
ReLU	음수는 0, 양수는 그대로	일반 DNN, CNN
LeakyReLU	음수도 조금 통과	Dead ReLU 방지
GELU	부드러운 ReLU 계열	Transformer, GPT, BERT
Softmax	확률 분포 출력	다중 분류 출력층

일반적인 은닉층에서는 ReLU 를 가장 많이 사용합니다.

`nn.ReLU()`

이진 분류 출력층에서는 Sigmoid 를 사용합니다.

`nn.Sigmoid()`

다중 분류에서는 Softmax 개념을 사용하지만, PyTorch 에서는 보통 CrossEntropyLoss 가 내부적으로 처리하므로 출력층에 Softmax 를 직접 붙이지 않는 경우가 많습니다.

5. 손실 함수(Loss Function)

손실 함수는 모델의 예측이 얼마나 틀렸는지 계산합니다.

예측값이 실제값과 비슷하면 손실값이 작고, 많이 틀리면 손실값이 큼니다.

회귀 문제

집값 예측처럼 숫자를 예측할 때는 보통 MSE 를 사용합니다.

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

PyTorch 코드:

```
criterion = nn.MSELoss()
```

이진 분류

합격/불합격, 정상/불량처럼 두 클래스를 분류할 때 사용합니다.

```
criterion = nn.BCELoss()
```

또는 더 안정적인 방식으로:

```
criterion = nn.BCEWithLogitsLoss()
```

다중 분류

숫자 0~9 분류, 동물 종류 분류처럼 여러 클래스를 분류할 때 사용합니다.

```
criterion = nn.CrossEntropyLoss()
```

6. 역전파(Backpropagation)

역전파는 딥러닝 학습의 핵심입니다.

순전파가 예측값을 만드는 과정이라면, 역전파는 오차의 원인을 뒤에서 앞으로 추적하는 과정입니다.

모델이 틀렸을 때 다음 질문을 합니다.

어떤 가중치가 오차를 크게 만들었는가?

어떤 편향을 수정해야 하는가?

각 층의 뉴런은 얼마나 책임이 있는가?

역전파는 손실값을 각 가중치에 대해 미분합니다.

$$\frac{\partial Loss}{\partial w}$$

이 값이 바로 Gradient 입니다.

Gradient 가 크면 해당 가중치가 오차에 큰 영향을 주었다는 뜻입니다.

7. 가중치 업데이트

역전파로 Gradient 를 계산한 후 Optimizer 가 가중치와 편향을 수정합니다.

가장 기본 공식은 다음과 같습니다.

$$w = w - learning_rate \times gradient$$

PyTorch 에서는 다음처럼 사용합니다.

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

의미는 다음과 같습니다.

optimizer.zero_grad() : 이전 학습에서 남아 있던 기울기를 초기화합니다.

loss.backward() : 손실값을 기준으로 모든 가중치와 편향의 gradient 를 계산합니다.

optimizer.step() : 계산된 gradient 를 이용하여 w 와 b 를 업데이트합니다.

8. Optimizer 의 종류

Optimizer 는 가중치를 어떻게 수정할지 결정하는 알고리즘입니다.

Optimizer	특징
SGD	가장 기본적인 경사하강법
Momentum	이전 이동 방향을 반영
RMSProp	학습률을 자동 조절
Adam	Momentum + RMSProp 장점 결합
AdamW	Adam 에 Weight Decay 개선

실무에서는 보통 Adam 또는 AdamW 를 많이 사용합니다.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

9. PyTorch 신경망 예제

아래는 가장 기본적인 다층 퍼셉트론 구조입니다.

```
import torch
import torch.nn as nn

# 신경망 모델 정의
class SimpleDNN(nn.Module):
    def __init__(self):
        super().__init__()

        # 입력층에서 첫 번째 은닉층으로 연결
        self.fc1 = nn.Linear(10, 64)

        # 첫 번째 은닉층에서 두 번째 은닉층으로 연결
        self.fc2 = nn.Linear(64, 32)

        # 두 번째 은닉층에서 출력층으로 연결
        self.fc3 = nn.Linear(32, 1)

        # 활성화 함수 정의
        self.relu = nn.ReLU()

    def forward(self, x):
        # 첫 번째 선형 변환 후 ReLU 적용
        x = self.relu(self.fc1(x))

        # 두 번째 선형 변환 후 ReLU 적용
        x = self.relu(self.fc2(x))

        # 출력층 계산
```

```
x = self.fc3(x)

return x

model = SimpleDNN()
print(model)
```

이 모델은 입력값 10 개를 받아 최종적으로 숫자 1 개를 출력합니다.

예를 들어 집값 예측, 매출 예측, 점수 예측 같은 회귀 문제에 사용할 수 있습니다.

10. 계층별 역할 정리

입력층
→ 원본 데이터를 받음

은닉층 1
→ 단순 패턴 학습

은닉층 2
→ 조합된 패턴 학습

은닉층 3 이상
→ 추상적 특징 학습

출력층
→ 최종 예측값 생성

이미지 예제로 보면 다음과 같습니다.

픽셀

- 선과 모서리
- 눈, 코, 입
- 얼굴 형태
- 사람/고양이/자동차 분류

텍스트 예제로 보면 다음과 같습니다.

단어

- 문법적 관계
- 문장 의미
- 문맥 이해
- 다음 단어 예측

11. 대표적인 딥러닝 신경망 알고리즘

DNN

가장 기본적인 완전연결 신경망입니다.

표 형태 데이터, 수치 데이터, 기본 분류/회귀 문제에 사용합니다.

Input → Linear → ReLU → Linear → ReLU → Output

CNN

이미지 처리에 강한 신경망입니다.

합성곱 계층을 사용하여 이미지의 공간적 특징을 학습합니다.

Image → Conv → ReLU → Pooling → Flatten → Linear

사용 예:

이미지 분류, 얼굴 인식, 의료 영상 분석

RNN

순서가 있는 데이터를 처리합니다.

이전 상태 + 현재 입력 → 현재 출력

사용 예:

시계열 예측, 문장 처리, 음성 데이터

LSTM / GRU

RNN의 장기 기억 문제를 개선한 구조입니다.

긴 문장이나 긴 시계열 데이터를 처리할 때 사용합니다.

Transformer

현재 자연어 처리와 생성형 AI 의 핵심 구조입니다.

Self-Attention 을 이용하여 문장 전체의 관계를 한 번에 파악합니다.

사용 예:

GPT, BERT, 번역기, 챗봇, 문서 요약

12. 딥러닝 학습 코드 기본 구조

```
for epoch in range(100):  
    # 1. 모델 예측  
    pred = model(X_train)  
  
    # 2. 손실 계산  
    loss = criterion(pred, y_train)  
  
    # 3. 이전 gradient 초기화  
    optimizer.zero_grad()  
  
    # 4. 역전파 수행  
    loss.backward()  
  
    # 5. 가중치와 편향 업데이트  
    optimizer.step()  
  
    # 6. 학습 상태 출력  
    print(epoch, loss.item())
```

이 구조는 거의 모든 PyTorch 딥러닝 코드의 기본입니다.

딥러닝 신경망은 입력층, 은닉층, 출력층으로 구성됩니다.

입력층은 데이터를 받고, 은닉층은 특징을 학습하며, 출력층은 최종 결과를 만듭니다.

학습 과정은 순전파로 예측값을 만들고, 손실 함수로 오차를 계산한 뒤, 역전파로 각 가중치의 책임을 계산하고, Optimizer 가 가중치와 편향을 수정하는 방식으로 반복됩니다.

즉 딥러닝은 다음 과정을 계속 반복하는 알고리즘입니다.

예측한다 → 틀린 정도를 계산한다 → 왜 틀렸는지 계산한다 → 조금 고친다

4. 활성화 함수

활성화 함수(Activation Function)는 신경망에서 입력값을 비선형(Non-linear)으로 변환하는 함수입니다.

만약 활성화 함수가 없다면 여러 층을 쌓아도 결국 하나의 선형회귀와 동일해집니다.

즉, 딥러닝이 복잡한 문제를 해결할 수 있는 가장 중요한 이유가 바로 활성화 함수입니다.

활성화 함수가 필요한 이유

퍼셉트론 계산식

$$z = \sum_{i=1}^n w_i x_i + b$$

여기서

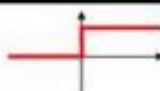
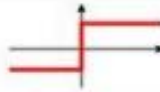
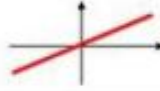
$$y = f(z)$$

를 적용합니다.

예)

$$z = 3.2$$

$$y = \text{activation}(z)$$

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer NN	

1. Step Function (계단 함수)

퍼셉트론 초창기 사용

수식

$$f(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

특징

- 단순
- AND, OR 구현 가능
- 미분 불가능

PyTorch

```
import torch

x = torch.tensor([-2,-1,0,1,2])

y = (x > 0).float()

print(y)
```

장점

- 매우 단순

단점

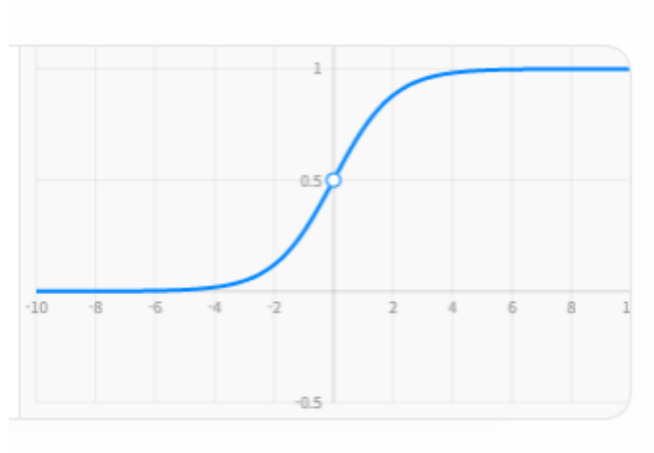
- 역전파 불가능

현재 사용 안 함

2. Sigmoid

가장 유명한 활성화 함수
수식

$$f(x) = \frac{1}{1+e^{-x}}$$



그래프 특징

0 ~ 1 사이 출력

S 자 형태

PyTorch

```
import torch
import torch.nn as nn

sigmoid = nn.Sigmoid()

x = torch.tensor([-2., -1., 0., 1., 2.])

print(sigmoid(x))
```

결과

```
0.119
0.269
0.500
0.731
0.881
```


장점

- 확률 표현 가능
- Binary Classification 출력층

단점

Gradient Vanishing

$$x \gg 0$$

또는

$$x \ll 0$$

이면

기울기가 거의 0

현재 사용

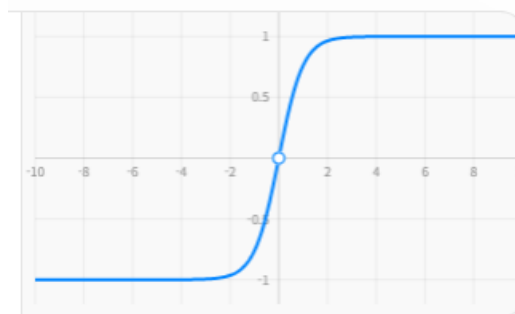
- 출력층(Binary Classification)

3. Tanh

Sigmoid 개선 버전

수식

$$f(x) = \tanh(x)$$



출력 범위

$$-1 \sim 1$$

PyTorch

```
tanh = nn.Tanh()
print(tanh(x))
```

결과

-0.964

-0.761

0

0.761

0.964

장점

- 중심이 0
- Sigmoid 보다 학습 빠름

단점

여전히 Gradient Vanishing 존재

현재

RNN 일부 사용

4. ReLU

현재 가장 많이 사용하는 함수

수식

$$f(x) = \max(0, x)$$

그래프

음수 $\rightarrow 0$

양수 $\rightarrow$ 그대로

PyTorch

```
relu = nn.ReLU()
```

```
print(relu(x))
```

결과

```
0
0
0
1
2
```

장점

- 계산 빠름
- Gradient Vanishing 거의 없음
- CNN, DNN 표준

단점

Dead ReLU

음수 영역에서

$$Gradient = 0$$

현재

가장 많이 사용

5. Leaky ReLU

Dead ReLU 개선

수식

$$f(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}$$

PyTorch

```
leaky = nn.LeakyReLU(
    negative_slope=0.01
)

print(leaky(x))
```

결과

-0.02
-0.01
0
1
2

장점

- Dead ReLU 해결

단점

- 추가 파라미터 존재

실무

- ReLU 대체 가능

6. PReLU

Leaky ReLU 개선

수식

$$f(x) = \begin{cases} x & x > 0 \\ ax & x \leq 0 \end{cases}$$

여기서

a

도 학습

PyTorch

```
prelu = nn.PReLU()
```

장점

자동 최적화

단점

파라미터 증가

7. ELU

Exponential Linear Unit

수식

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

PyTorch

```
elu = nn.ELU()
```

장점

- 평균 0 근처
- 학습 안정적

단점

계산량 증가

8. SELU

Self Normalizing

특징

자동 정규화 효과

PyTorch

```
selu = nn.SELU()
```

사용

Self-Normalizing Network

9. GELU

Transformer 표준

Google BERT

OpenAI GPT

사용

수식

$$GELU(x) = xP(X \leq x)$$

PyTorch

```
gelu = nn.GELU()
```

장점

- 매우 부드러움
- Transformer 최적

현재

LLM 표준

10. Swish

Google 개발

수식

$$f(x) = x\sigma(x)$$

PyTorch

```
swish = nn.SiLU()
```

장점

ReLU 보다 성능 우수

11. Mish

수식

$$f(x) = x \tanh(\text{softplus}(x))$$

PyTorch

```
mish = nn.Mish()
```

특징

매우 부드러운 함수

활성화 함수 성능 비교

함수	출력범위	Vanishing	속도	현재 사용
Step	0~1	심함	빠름	사용 안 함
Sigmoid	0~1	심함	보통	출력층
Tanh	-1~1	있음	보통	일부 RNN
ReLU	0~ ∞	거의 없음	매우 빠름	가장 많이 사용
Leaky ReLU	$-\infty \sim \infty$	없음	빠름	자주 사용
ELU	$-\alpha \sim \infty$	적음	보통	일부 사용
SELU	$-\infty \sim \infty$	적음	보통	특정 모델
GELU	$-\infty \sim \infty$	없음	느림	GPT/BERT
Swish(SiLU)	$-\infty \sim \infty$	없음	보통	EfficientNet
Mish	$-\infty \sim \infty$	없음	보통	연구용

실무 추천

일반 DNN : nn.ReLU()

CNN : nn.ReLU()

Dead ReLU 발생 : nn.LeakyReLU()

Transformer : nn.GELU()

GPT/BERT/LLM : nn.GELU()

Binary Classification 출력층 : nn.Sigmoid()

Multi-Class Classification 출력층 : nn.Softmax(dim=1)

현재 딥러닝 실무에서 가장 많이 사용되는 활성화 함수는 **ReLU**, **LeakyReLU**, **GELU**, **Softmax**, **Sigmoid**이며, 특히 GPT·BERT 계열 Transformer 모델은 대부분 **GELU**를 사용합니다.